



**Submitted By**

Basmala Samir

Evronia Romany

Malak Ibrahim

Asmaa Ashraf

**Digital Egypt Pioneers Initiative (DEPI)**

2026

# Table of Contents

|                                                          |    |
|----------------------------------------------------------|----|
| 1. Executive Summary .....                               | 4  |
| 2. Project Overview .....                                | 5  |
| 3. Problem Statement .....                               | 6  |
| 4. Objectives .....                                      | 7  |
| 5. Scope .....                                           | 8  |
| 5.1 In Scope .....                                       | 8  |
| 5.2 Out of Scope .....                                   | 8  |
| 6. Literature Review .....                               | 9  |
| 6.1 Background .....                                     | 9  |
| 6.2 Dimensional Modelling and the Galaxy Schema .....    | 9  |
| 6.3 Rule-Based Financial Health Monitoring .....         | 9  |
| 6.4 Generative AI for Financial Advisory .....           | 9  |
| 7. System Architecture .....                             | 10 |
| 7.1 Technical Workflow .....                             | 11 |
| 7.2 Layered Responsibilities .....                       | 11 |
| 8. Pipeline Architecture (Bronze and Silver) .....       | 12 |
| 8.1 Bronze Layer .....                                   | 12 |
| 8.1.1 Sources Ingested .....                             | 12 |
| 8.1.2 Implementation Approach .....                      | 12 |
| 8.1.3 Business Value and Best Practices .....            | 1  |
| 8.2 Silver Layer .....                                   | 13 |
| 8.2.1 A Shared Date-Parsing Strategy .....               | 13 |
| 8.2.2 Standardizing Mathematical Direction .....         | 13 |
| 8.2.3 Eliminating Redundant Records .....                | 13 |
| 8.2.4 Deriving New Columns .....                         | 13 |
| 9. Pipeline Architecture (Gold) .....                    | 14 |
| 9.1 Galaxy Schema .....                                  | 14 |
| 9.1.1 Model Inventory .....                              | 15 |
| 9.1.2 How the Dimensions Connect the Facts .....         | 15 |
| 9.2 Gold Layer .....                                     | 16 |
| 9.2.1 Dimension Tables .....                             | 16 |
| 9.2.1.1 dim_date .....                                   | 16 |
| 9.2.1.2 dim_accounts .....                               | 16 |
| 9.2.1.3 dim_categories .....                             | 16 |
| 9.2.1.4 dim_employee .....                               | 17 |
| 9.2.1.5 dim_product .....                                | 17 |
| 9.2.2 Fact Tables .....                                  | 17 |
| 9.2.2.1 fact_transactions .....                          | 17 |
| 9.2.2.2 fact_monthly_summary .....                       | 17 |
| Shortage-Detection Classification Logic .....            | 18 |
| 9.2.2.3 fact_sales .....                                 | 19 |
| 9.2.2.4 fact_payroll .....                               | 19 |
| 10. Business Views .....                                 | 20 |
| 10.1 Views Feeding the Executive Overview Page .....     | 21 |
| 10.2 Views Feeding the Payroll & Expenses Page .....     | 21 |
| 10.3 View Feeding the Sales Page .....                   | 21 |
| 10.4 View Feeding the Business Health Monitor Page ..... | 22 |
| 10.5 The Financial Insights Page .....                   | 22 |
| 11. Power BI Dashboards .....                            | 23 |
| 11.1 Executive Overview Page .....                       | 23 |

|                                                                    |    |
|--------------------------------------------------------------------|----|
| 11.2 Sales Page .....                                              | 23 |
| 11.3 Payroll & Expenses Page .....                                 | 24 |
| 11.4 Business Health Monitor Page .....                            | 25 |
| 11.5 Financial Insights Page .....                                 | 26 |
| 12. AI Suggestions .....                                           | 27 |
| 12.1 How the Workflow Identifies Which Months Need Analysis .....  | 27 |
| 12.2 Assembling the Financial Context for Each Flagged Month ..... | 27 |
| 12.3 Prompting Strategy .....                                      | 28 |
| 12.4 Technology Used .....                                         | 28 |
| 12.5 What the Workflow Produces .....                              | 28 |
| 13. Website .....                                                  | 29 |
| 14. Future Enhancements .....                                      | 30 |
| 15. Conclusion .....                                               | 30 |

# 1. Executive Summary

SteadyFin is a cash-flow intelligence system built for independent coffee shop business. The system was designed to solve a problem that is common to small businesses: financial visibility is fragmented across multiple accounts and systems, and by the time a cash shortage becomes obvious, it is often too late to react to it.

The project ingests two years of operational and financial history from five independent data sources — a primary checking account, a secondary checking account used for payroll transfers, a point-of-sale (POS) sales export, a credit card account, and a payroll export from Gusto — and consolidates them into a single governed lakehouse built on a Bronze, Silver, and Gold medallion architecture. The Gold layer is modelled as a galaxy (fact-constellation) schema in which multiple fact tables share a common set of conformed dimensions, most notably a single calendar dimension that ties transactions, sales, payroll, and monthly summaries together on a consistent time axis.

On top of the Gold layer, SteadyFin implements an automated shortage-detection engine that classifies every month of the business's history into a financial health state — Healthy, Warning, Expense Spike, Income Drop, Critical, or Startup Period — using rolling three-month baselines rather than fixed thresholds. Months that are flagged are automatically routed to an AI advisory layer, built on a generative language model, which is provided with the full financial context for that month (KPIs, expense breakdown, cash runway, top vendors, top-selling products, and payroll detail) and asked to produce a structured, data-grounded diagnosis rather than generic financial advice.

The curated Gold-layer views feed a four-page Power BI reporting suite covering an executive overview, a sales, business health monitor and product performance view, and a payroll and expense view, giving business owners and consultants a single place to monitor the health of the business. A website channel exposes the same insights to end users outside of the Power BI environment.

This document describes the full engineering lifecycle of SteadyFin, from project planning through to the design of every transformation in the Bronze, Silver, and Gold layers, the construction of the galaxy schema, the business views that power the dashboards, the AI recommendation workflow, and the reasoning behind each architectural decision.

## 2. Project Overview

---

A Coffee shop operate on thin margins and irregular cash cycles: daily sales income arrives continuously through a POS system, while expenses such as payroll, supplier payments, marketing, and utilities are paid on their own independent schedules through a mix of checking accounts and a credit card. Without a consolidated financial picture, an owner typically has to check several disconnected statements before they can answer a simple question such as "can I make payroll next month" or "why did my balance drop this quarter."

SteadyFin was built to close that gap. It is a full data engineering pipeline — not a single dashboard — that takes raw exports from the coffee shop's real financial systems, cleans and standardizes them, models them into a business-ready analytical layer, and layers automated risk detection and AI-generated advice on top. The project therefore spans the complete spectrum of a modern analytics stack: ingestion, transformation, dimensional modelling, semantic views, business intelligence, and applied generative AI.

### 3. Problem Statement

---

Small food-and-beverage businesses such as coffee shops rarely have a dedicated finance team. Cash movements are spread across a primary checking account, a secondary account reserved for payroll transfers, a credit card used for supplier and marketing spend, and a point-of-sale system that records every item sold. Each of these systems produces its own export, in its own format, on its own schedule, and none of them talk to one another.

This fragmentation creates three concrete problems that SteadyFin was designed to address:

1. Lack of a single source of truth. An owner cannot easily see, in one place, how much cash the business generated in a month, how much it spent, and what it has left, because that information is split across at least four separate account statements.
2. No early warning system. Cash shortages are typically discovered only after they have already happened — for example, when a payment fails or a balance drops unexpectedly — rather than being anticipated from the trend of the preceding months.
3. No data-grounded guidance for corrective action. Even when an owner suspects a problem, generic financial advice ("cut costs", "increase sales") is not actionable without knowing which specific expense category, vendor, or payroll line is actually responsible for the deviation.

SteadyFin's data engineering pipeline, dimensional model, and AI advisory layer were designed specifically to remove these three gaps, turning five disconnected raw exports into a governed, queryable, and explainable financial monitoring system.

## 4. Objectives

---

The following objectives guided the design and implementation of every layer of the SteadyFin pipeline:

- Consolidate five independent data sources — the two checking accounts, the credit card account, the POS sales export, and the payroll export — into a single governed lakehouse rather than analyzing them in isolation.
- Standardize and cleanse raw transactional data so that dates, transaction signs, and duplicate records are handled consistently across every account before any business logic is applied.
- Model the cleansed data into a business-ready, query-optimized galaxy schema with conformed dimensions, so that transactions, sales, and payroll can all be analyzed on the same calendar axis.
- Design an automated, rolling-baseline shortage-detection mechanism that classifies each month's financial health without relying on a single hard-coded threshold that would misfire during the business's early months.
- Expose the Gold layer through a set of purpose-built SQL views, each aligned to a specific dashboard page, so that the BI layer never has to re-derive business logic that the data engineering layer has already resolved.
- Deliver a five-page interactive Power BI reporting suite giving stakeholders visibility into executive KPIs, sales and product performance, and payroll and expense composition.
- Integrate a generative AI advisory step that is grounded exclusively in the Gold-layer data for a flagged month, producing a structured and explainable recommendation instead of generic financial commentary.
- Provide a website channel through which the insights generated by the pipeline can be surfaced to end users.

## 5. Scope

---

The scope of SteadyFin covers the complete path from raw source files to AI-generated financial recommendations.

### 5.1 In Scope

- Ingestion of five raw CSV exports (two checking accounts, one credit card account, one POS sales export, one payroll export) into a Bronze layer with no transformation.
- Cleansing, type standardization, date normalization, deduplication, and derivation of the signed cash-flow amount in the Silver layer.
- Construction of a Gold-layer galaxy schema comprising five dimension tables and four fact tables.
- Nine business-ready SQL views mapped directly to the four pages of the reporting suite (Executive Overview, Expense Deep-Dive, Sales & Products, AI Recommendations).
- A rule-based monthly shortage-detection engine using rolling three-month baselines.
- A five-page Power BI dashboard (Executive Overview, Sales Dashboard, Payroll & Expenses).
- An AI advisory workflow that queries flagged months and produces structured JSON recommendations.
- A website channel presenting the pipeline's insights to end users.

### 5.2 Out of Scope

- Real-time or streaming ingestion — all data is processed as historical batch loads covering 2022–2023.
- Direct bank or POS system API integration — the pipeline consumes CSV exports rather than connecting live to external financial systems.
- Multi-location or multi-tenant support — the model represents a single coffee shop business.



## 6. Literature Review

---

### 6.1 Background

Small retail and food-service businesses have historically relied on manual bookkeeping or disconnected spreadsheet exports from their bank, credit card provider, point-of-sale system, and payroll processor. The absence of a unified data layer means that trend analysis, anomaly detection, and forward-looking cash planning are difficult to perform consistently, and are usually left to whatever a bookkeeper can reconstruct manually at the end of a reporting period.

The medallion architecture pattern — organizing data into progressively refined Bronze (raw), Silver (cleansed), and Gold (business-ready) layers — has become a widely adopted approach in modern lakehouse platforms because it separates the concerns of raw data preservation, data quality enforcement, and business-level modelling into distinct, independently testable stages. SteadyFin adopts this pattern directly, implementing each layer as its own notebook with its own database namespace within the Databricks lakehouse.

### 6.2 Dimensional Modelling and the Galaxy Schema

Where a single fact table is sufficient to answer most questions, a star schema is the conventional choice. SteadyFin, however, needs to answer questions that span more than one business process at once — bank and credit card transactions, monthly financial summaries, POS sales, and payroll — and these processes do not all share the same grain. A galaxy (fact-constellation) schema, in which several fact tables are linked through a set of shared, conformed dimensions, is the appropriate modelling pattern for this situation, and it is the pattern implemented in the Gold layer of this project.

### 6.3 Rule-Based Financial Health Monitoring

Rather than relying on a single static threshold (for example, "flag any month where the balance falls below \$X"), SteadyFin's shortage-detection logic compares each month against a rolling three-month trailing average of its own revenue, expenses, and balance. This approach is more robust to the natural seasonality and growth trajectory of a young business, because it evaluates a month relative to its own recent history rather than an arbitrary fixed number that would need to be manually re-tuned as the business grows.

### 6.4 Generative AI for Financial Advisory

Large language models are increasingly used to translate structured analytical output into natural-language explanations and recommendations. The risk in this application is that a model asked to comment on "cash flow problems" in the abstract will produce generic, non-actionable advice. SteadyFin addresses this by supplying the model with the complete, concrete financial context for a specific flagged month — not just the headline numbers, but the expense breakdown, vendor-level spend, product performance, and payroll detail behind them — and by constraining its output to a strict JSON schema that forces it to name a root cause and prioritized recommendations rather than write a general essay.

## 7. System Architecture

SteadyFin follows a medallion lakehouse architecture, implemented entirely on a Databricks workspace using Delta Lake as the storage format. The architecture is organized into three progressively refined data layers — Bronze, Silver, and Gold — each isolated in its own database namespace under a shared catalog (cash\_flow\_project), followed by a BI layer that consumes the Gold-layer outputs.

This separation of concerns is a deliberate architectural decision rather than an implementation convenience. Keeping raw, cleansed, and business-ready data physically separated means that a downstream defect can always be root-caused by walking backward one layer at a time, and that any layer can be rebuilt from its upstream source without risking the integrity of the layers below it.

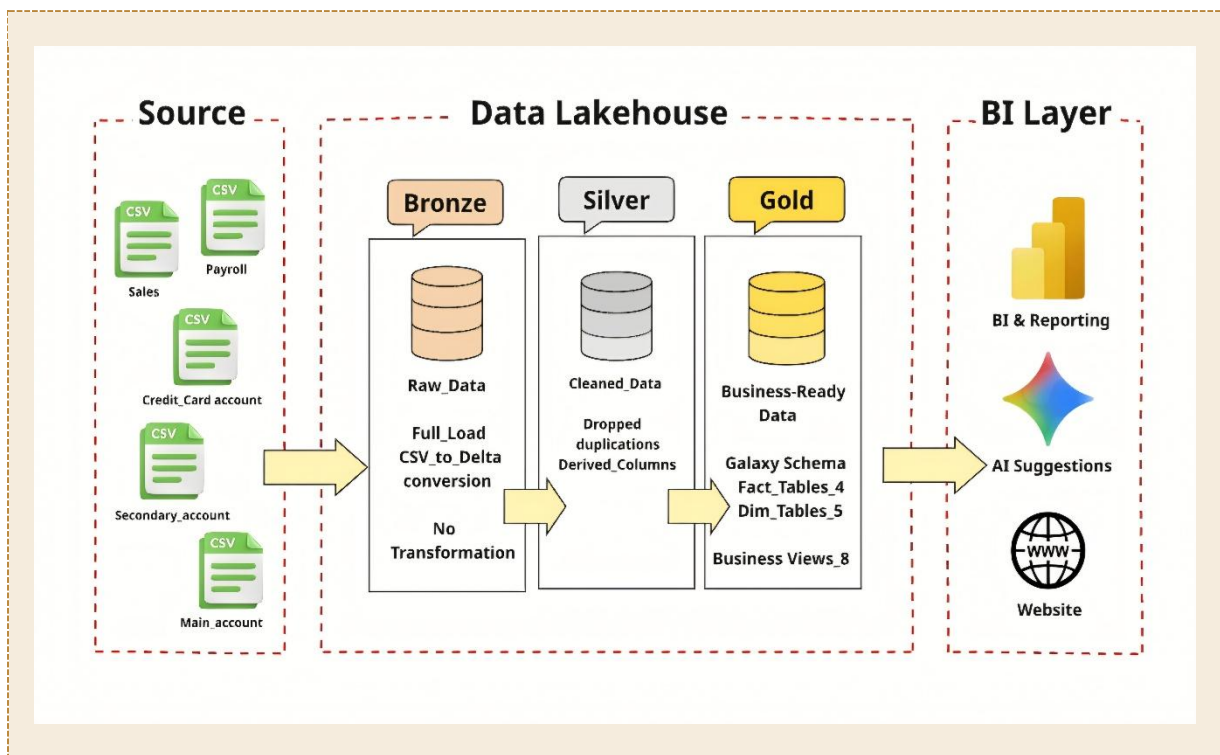


Figure 7.1 — SteadyFin lakehouse architecture: Source → Bronze → Silver → Gold → BI Layer

## 7.1 Technical Workflow

| ID   | Requirement                                                                                                                                                   |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TW-1 | The system ingests five raw CSV sources (checking main, checking secondary, credit card, POS sales, payroll) into a Bronze layer without transformation.      |
| TW-2 | The system standardizes date formats, derive calendar attributes, compute a signed transaction amount, and remove duplicate transactions in the Silver layer. |
| TW-3 | The system builds conformed dimension tables for date, account, category, employee, and product.                                                              |
| TW-4 | The system builds fact tables for ledger transactions, monthly financial summary, POS sales, and payroll.                                                     |
| TW-5 | The system classifies every month into a financial health state using a rolling three-month baseline.                                                         |
| TW-6 | The system exposes business logic through SQL views aligned one-to-one with dashboard pages.                                                                  |
| TW-7 | The system presents KPIs and trends through a multi-page Power BI dashboard.                                                                                  |
| TW-8 | The system automatically generates a structured, data-grounded recommendation for any month flagged as needing attention.                                     |
| TW-9 | The system surfaces insights to end users through a website channel.                                                                                          |

## 7.2 Layered Responsibilities

| Layer    | Responsibility                                                                                                                         | Database Namespace                 |
|----------|----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| Bronze   | Raw, full-load ingestion of the five source CSV files with no transformation applied                                                   | cash_flow_project.cash_flow_bronze |
| Silver   | Type standardization, date normalization, deduplication, and derivation of the signed cash-flow amount                                 | cash_flow_project.cash_flow_silver |
| Gold     | Dimensional modelling into a galaxy schema, monthly aggregation, and shortage-detection business logic                                 | cash_flow_project.cash_flow_gold   |
| BI Layer | Power BI dashboards, the AI recommendation workflow, and the website channel, all reading exclusively from Gold-layer views and tables | consumption layer                  |

## 8. Pipeline Architecture ( Bronze and Silver )

### 8.1 Bronze Layer

The Bronze layer is the entry point of the SteadyFin pipeline. Its sole purpose is to bring the five raw CSV exports produced by the coffee shop's external systems — its two checking accounts, its credit card provider, its point-of-sale system, and its payroll processor (Gusto) — into the lakehouse as-is, with no cleansing, no renaming, and no business logic applied.

#### 8.1.1 Sources Ingested

| Source File                    | Bronze Table               | Business Meaning                                                                            |
|--------------------------------|----------------------------|---------------------------------------------------------------------------------------------|
| checking_account_main.csv      | checking_accounts_main     | Primary operating bank account — day-to-day revenue deposits and operating expense payments |
| checking_account_secondary.csv | checking_account_secondary | Secondary checking account used exclusively to hold and transfer payroll funds              |
| coffee_sales_2years.csv        | coffee_shop_sales          | Two years of item-level point-of-sale transactions from the coffee shop                     |
| credit_card_account.csv        | credit_card_account        | Business credit card used for supplier, marketing, and utility payments                     |
| gusto_payroll.csv              | gusto_payroll              | Payroll export from Gusto covering employee and contractor payments                         |

#### 8.1.2 Implementation Approach

A dedicated Bronze database (cash\_flow\_bronze) is created first, keeping raw data physically isolated from every other layer. Each of the five source files is then read independently with header inference and automatic schema inference enabled, so that Spark determines the column names and data types directly from the CSV rather than requiring a manually maintained schema definition at this earliest stage. Each resulting DataFrame is written out as a managed Delta table using an overwrite write mode, under its own name inside the Bronze database.

#### 8.1.3 Business Value and Best Practices

- **Auditability:** raw source data is preserved unmodified, giving the team a verifiable record of exactly what each source system reported.
- **Separation of concerns:** no business logic is mixed into the ingestion stage, keeping the Bronze notebook simple, predictable, and easy to re-run.
- **Schema flexibility:** inferred schemas at the Bronze stage avoid the need to hand-maintain five separate schema definitions before any of the data has even been explored.
- **Idempotency:** overwrite-mode writes mean the Bronze layer can be safely re-executed at any time without manual cleanup.

## 8.2 Silver Layer

The Silver layer is where SteadyFin's five raw Bronze tables are turned into trustworthy, analysis-ready data. Every table in this layer lives in its own database, `cash_flow_silver`, and every transformation applied here follows the same underlying goal: remove the inconsistencies that exist naturally between five independently produced source systems, without yet imposing any cross-source business modelling. That modelling is reserved for the Gold layer.

Three problems recur across the raw data and are solved once, consistently, in this layer: inconsistent date formats, the need for a signed transaction amount that behaves correctly in cash-flow arithmetic, and duplicate transaction records.

### 8.2.1 A Shared Date-Parsing Strategy

Because the five source systems were produced independently — a bank export, a secondary bank export, a point-of-sale system, a credit card statement, and a payroll processor — their date columns do not all follow the same textual format. Rather than writing bespoke date-parsing logic five times, the Silver layer defines a single reusable helper that attempts to parse each date column against two known formats (ISO-style `yyyy-MM-dd` and US-style `M/d/yyyy`), returning whichever one succeeds first.

This is a deliberate maintainability decision: if a sixth source were added in the future with yet another date format, only this one helper function would need to be extended, rather than hunting down every place a date is parsed across the pipeline.

### 8.2.2 Standardizing Mathematical Direction

In their raw form, records from bank accounts, credit cards, and sales systems suffer from transactional direction inconsistencies. Most bank and credit card systems list all transaction values as absolute positive numbers, relying on a separate descriptive column like "Debit" or "Credit" to indicate whether money came in or went out. To prevent critical calculation errors downstream, the Silver layer creates a uniform `signed_amount` column. It automatically attaches a minus sign (—) to cash outflows (such as vendor bills, credit card spending, and employee payroll taxes) while keeping incoming deposits and customer sales positive. This ensures that the downstream engine can accurately add up the total cash position without manual accounting errors.

### 8.2.3 Eliminating Redundant records

When raw data files are exported multiple times or re-imported during pipeline runs, identical transactions can accidentally get recorded twice, which artificially inflates bank balances and sales numbers. The Silver layer cleans this up by running automated deduplication checks across all incoming datasets. It drops exact copies of transactions based on unique transaction IDs.

### 8.2.4 Deriving New Columns

Raw timestamps are often trapped as complex text strings or recorded at a very granular hour-and-minute level, making data sorting and monthly grouping extremely slow for dashboard queries. The Silver layer fixes this by automatically breaking down every validated date into standard, separate columns for **Year**, **Month**, **Day**, and **Quarter**. By calculating these calendar attributes once up front in this layer, downstream financial business views and Power BI charts can slice, dice, and filter data instantly without re-processing long date strings every time a page loads.

## 9. Pipeline Architecture ( Gold )

### 9.1 Galaxy Schema

SteadyFin's Gold layer is deliberately modelled as a galaxy schema — also known as a fact constellation — rather than a single star schema. A galaxy schema is the appropriate pattern here because the business has more than one distinct process to measure (bank and credit card transactions, monthly financial performance, point-of-sale sales, and payroll), and these processes do not share a single grain. Instead of forcing all of them into one oversized fact table, each process gets its own fact table, and the model's coherence comes from the dimensions that multiple fact tables share.

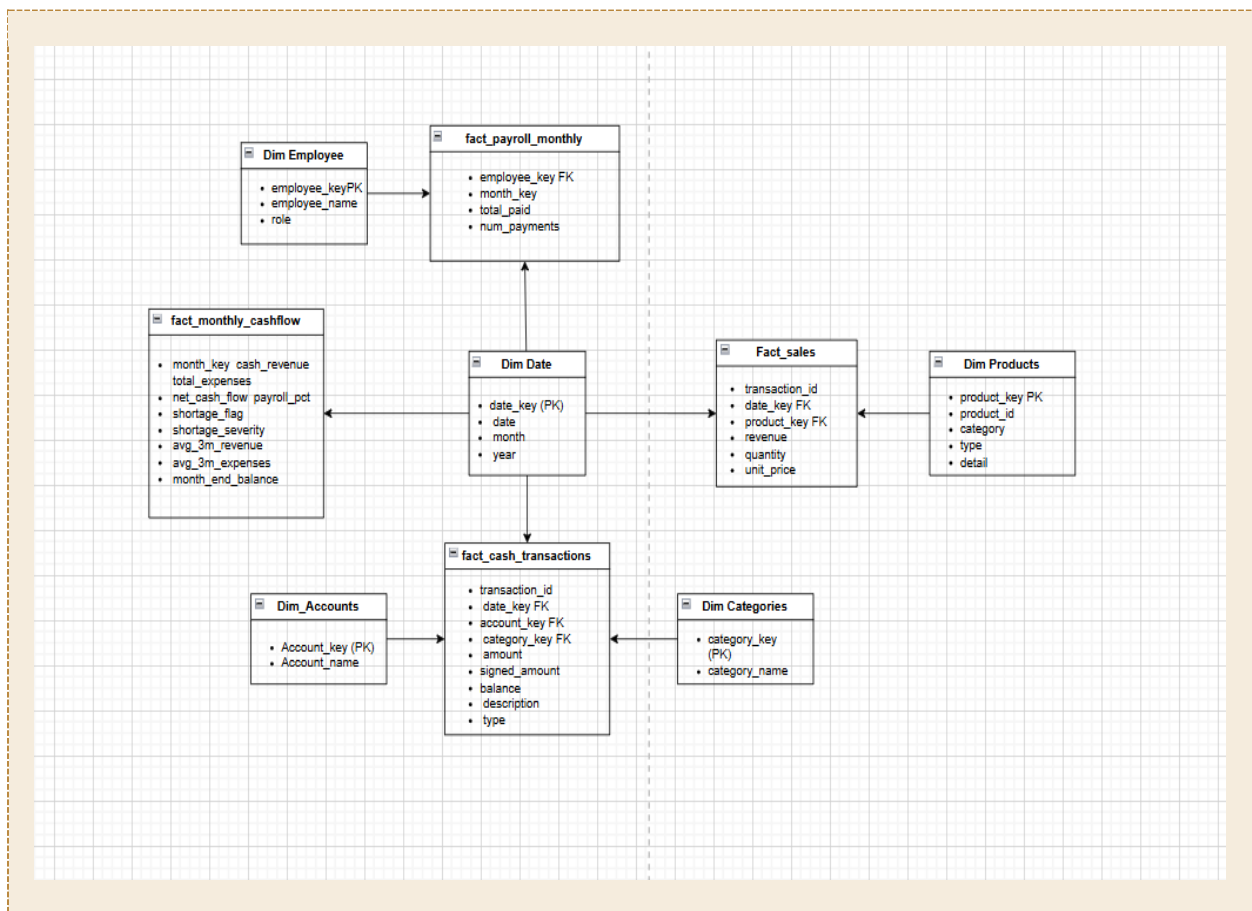


Figure 9.1 — SteadyFin Gold-layer galaxy schema: 5 dimension tables and 4 fact tables

### 9.1.1 Model Inventory

The complete Gold-layer model consists of five dimension tables and four fact tables:

| Dimension Table      | Grain / Content                                                                                                            |
|----------------------|----------------------------------------------------------------------------------------------------------------------------|
| dim_date             | One row per calendar day, 2022-01-01 to 2023-12-31, with date_key, calendar attributes, quarter, month_key, and is_weekend |
| dim_accounts         | One row per financial account: Checking Main, Checking Secondary, Credit Card                                              |
| dim_categories       | One row per business category: 9 categories spanning Income, Expense, Transfer, and Other                                  |
| dim_employee         | One row per unique employee or contractor, derived from payroll data                                                       |
| dim_product          | One row per unique menu item, derived from POS sales data, with a standardized average unit price                          |
| Fact Table           | Grain / Content                                                                                                            |
| fact_transactions    | One row per ledger transaction, across all 3 money accounts (2 checking + credit card)                                     |
| fact_monthly_summary | One row per calendar month, combining revenue, expenses, payroll, KPIs, rolling baselines, and shortage classification     |
| fact_sales           | One row per point-of-sale transaction                                                                                      |
| fact_payroll         | One row per employee, per month                                                                                            |

### 9.1.2 How the Dimensions Connect the Facts

dim\_date is the central, conformed dimension of the model: it is joined to fact\_transactions, fact\_monthly\_summary (via month\_key), fact\_sales, and — indirectly, through the month\_key it defines — fact\_payroll. Because every fact table can be sliced by the same calendar, a question that spans two processes at once (for example, "did marketing spend from the credit card correlate with a sales increase the same month") can be answered by joining fact\_transactions and fact\_sales through dim\_date, without needing a purpose-built table for that one question.

dim\_accounts and dim\_categories connect specifically to fact\_transactions, giving every ledger line a readable account name and a consistent business category regardless of which of the three money accounts it originated from. dim\_product connects to fact\_sales, letting every sale be grouped by category, type, or specific product. dim\_employee connects to fact\_payroll, letting payroll cost be analyzed by employee, role, or pay type.

## 9.2 Gold Layer

The Gold layer is where SteadyFin stops being a collection of cleansed source tables and becomes a genuine business analytics model. It lives in its own database, `cash_flow_gold`, and is built in two stages: first, five conformed dimension tables are constructed; second, four fact tables are built by joining the Silver-layer data against those dimensions and, in the case of the monthly summary fact, deriving the KPIs and shortage-detection logic that the rest of the system depends on.

Every Gold-layer table is written with Delta's overwrite mode and, where the schema evolves between iterations, the `overwriteSchema` option, keeping the Gold layer fully reproducible from the Silver layer beneath it at any time.

### 9.2.1 Dimension Tables

Five dimension tables were built, each representing a distinct, reusable business concept that more than one fact table needs to reference:

#### 9.2.1.1 `dim_date`

`dim_date` is the backbone of the entire Gold layer. It is generated programmatically as one row per calendar day across the full two-year project horizon (2022-01-01 through 2023-12-31), rather than being derived from any single source table. Each row carries an integer `date_key` (formatted as `yyyyMMdd`) for efficient joining, the raw date itself, and a full set of calendar attributes: year, month, day, quarter, a readable month name, a readable day name, a `month_key` formatted as `yyyy-MM` for monthly grouping, and an `is_weekend` flag.

Generating this dimension independently of any transactional source — rather than deriving it from, say, the distinct dates that appear in the sales table — guarantees that every calendar day in the two-year window exists in the dimension even if a particular day happened to have zero transactions in one of the source systems. This is what allows `dim_date` to act as a conformed dimension: every one of the four fact tables joins to the same calendar, so a month-over-month comparison in the sales fact and a month-over-month comparison in the cash-flow fact are always referring to exactly the same set of months.

#### 9.2.1.2 `dim_accounts`

`dim_accounts` is a small, manually defined lookup of the three money-holding accounts in the business: Checking Main and Checking Secondary (both of type Checking) and Credit Card (of type Credit Card). Because there are only three accounts and they do not change over time, this dimension is defined directly rather than derived from a source table, giving each account a stable surrogate `account_key` that the transaction fact table can reference.

#### 9.2.1.3 `dim_categories`

`dim_categories` defines the nine spending and income categories used to classify every transaction in the business: Sales Revenue (Income), COGS, Operating Expense, Payroll, Marketing, Utilities, and Supplies & COGS (all Expense), Internal Transfer (Transfer), and Other. Like `dim_accounts`, this dimension is small and manually defined, since the set of categories the business uses is fixed and known in advance. Having a single, explicit category dimension — rather than letting each source system supply its own free-text category label — is what makes it possible to compare, for example, credit-card-derived Marketing spend against Marketing spend from the checking account, under one consistent label.



#### 9.2.1.4 dim\_employee

dim\_employee is derived from the Silver-layer payroll data: the distinct combination of employee id, name, role, pay type, and paying account is extracted, and a surrogate employee\_key is generated using a dense rank ordered by employee id. Deriving this dimension from the payroll data itself, rather than defining it manually, means the employee list always stays in sync with whoever actually appears in a payroll run, without requiring a separately maintained HR table.

#### 9.2.1.5 dim\_product

dim\_product is derived from the Silver-layer POS sales data by grouping on product id, category, type, and detail, and computing each product's average unit price as its standard\_unit\_price — since a product's price can vary slightly across transactions (for example, due to promotions), the average gives a single representative price for the dimension. A surrogate product\_key is again generated with a dense rank ordered by product id, giving every unique menu item a stable key for the sales fact table to reference.

### 9.2.2 Fact Tables

Four fact tables were built, each representing a distinct business process at its own natural grain, and each linked back to the shared dimensions described above.

#### 9.2.2.1 fact\_transactions — every ledger line from all three money accounts

fact\_transactions is the finest-grained fact table in the model: one row per individual ledger transaction across all three money-holding accounts. Building it requires reconciling three sources that do not share an identical schema. The two checking accounts already carry a category column, but the credit card statement does not — it only records a vendor name. To bring the credit card into the same fact table, its vendor names are mapped explicitly to a business category (for example, Facebook Ads and Local Print Shop both map to Marketing, Utility Company maps to Utilities, Coffee Supplier maps to Supplies & COGS, and Payment to CC maps to Internal Transfer, with any unmapped vendor falling back to Other).

Once each of the three ledgers is tagged with its account name and given a consistent set of columns, they are combined into a single table and joined against dim\_date, dim\_accounts, and dim\_categories to attach the appropriate surrogate keys. Any transaction whose category could not be matched to a known category name is defaulted to category\_key 109 (Other) rather than left null, ensuring that every row in the fact table can always be grouped and filtered by category without special-case handling downstream.

#### 9.2.2.2 fact\_monthly\_summary — the financial health engine

fact\_monthly\_summary is the most business-critical table in the Gold layer: one row per calendar month, combining figures from all three financial sources and the payroll export, and layering the shortage-detection logic on top. It is built in four stages.

First, three separate monthly aggregates are computed independently: monthly\_checking sums the main checking account's revenue (Sales Revenue category), COGS expense, and operating expense for each month, and captures the last recorded balance of the month as the month-end balance. monthly\_cc sums credit card spend (debit transactions only, so that transfers back to pay off the card are excluded from spend), broken down into total spend, marketing spend, utilities spend, and supplies spend. monthly\_payroll sums total payroll, split between employee pay and contractor pay.

Second, these three monthly aggregates are joined together on month\_key into a single row-per-month table, and any month with no activity in a given source is filled with zero rather than left null. From this combined

base, the core KPIs are derived: `total_expenses` (the sum of COGS, operating expense, payroll, and credit card spend), `net_cash_flow` (revenue minus total expenses), and `payroll_pct_of_revenue` (total payroll as a percentage of revenue, only computed where revenue is positive). Quarter and calendar year are also derived from the month key at this stage, specifically to support quarter-over-quarter and year-over-year comparisons in the Gold-layer views.

Third, a set of rolling three-month trailing averages is computed using a Spark window function that looks back over the three months preceding — but not including — the current month: `avg_3m_revenue`, `avg_3m_expenses`, and `avg_3m_balance`. Using a trailing window rather than a fixed historical average means these baselines move with the business as it grows, rather than staying anchored to its very first months forever. A `month_number` column is also derived at this stage, simply counting each month's position in the sequence from the start of the data.

Fourth, the shortage-detection classification is applied, along with a numeric `shortage_severity` ranking and a `needs_ai_suggestion` boolean flag that determines whether a given month should be routed to the AI advisory workflow.

### Shortage-Detection Classification Logic

Rather than a single pass/fail threshold, every month is evaluated in order against six mutually exclusive rules, and the first rule that matches becomes that month's `shortage_flag`. Evaluating the rules in this specific priority order matters: it guarantees, for instance, that a month with genuinely negative cash flow is always classified as CRITICAL even if it also happens to have an unusually large expense relative to its own recent average, rather than being mislabeled with a lower-severity flag.

| Priority    | Flag           | Trigger Condition                                            | Meaning                                                         |
|-------------|----------------|--------------------------------------------------------------|-----------------------------------------------------------------|
| 1 (highest) | CRITICAL       | Net cash flow for the month is negative                      | The business spent more than it earned this month               |
| 2           | STARTUP_PERIOD | Month is among the first three months of recorded history    | Not enough trailing history yet for a reliable 3-month baseline |
| 3           | WARNING        | Month-end balance is below 1.5x the 3-month average expenses | Cash buffer is thin relative to typical monthly spend           |
| 4           | EXPENSE_SPIKE  | Total expenses exceed 1.2x the 3-month average expenses      | Spend this month is unusually high versus recent history        |
| 5           | INCOME_DROP    | Revenue falls below 0.8x the 3-month average revenue         | Sales this month are unusually low versus recent history        |
| 6 (default) | HEALTHY        | None of the above conditions are met                         | No shortage risk detected                                       |

A numeric `shortage_severity` (4 for CRITICAL, 3 for EXPENSE\_SPIKE, 2 for INCOME\_DROP, 1 for WARNING, 0 otherwise) is derived alongside the flag so that months can be ranked by urgency, and a `needs_ai_suggestion` boolean is set to true for any month flagged CRITICAL, EXPENSE\_SPIKE, INCOME\_DROP, or WARNING — the four states that represent a genuine, actionable risk, as opposed to HEALTHY or the STARTUP\_PERIOD grace window.

The STARTUP\_PERIOD state deserves particular attention as a design decision: without it, a business's first one or two months, which have no three-month trailing average to compare against, would either break the

rolling-average calculation or be compared against an artificially thin baseline, very likely producing a false CRITICAL or WARNING alarm before the business has even established a normal pattern of activity. Introducing an explicit grace state avoids penalizing the business for simply being new.

#### **9.2.2.3 fact\_sales — one row per point-of-sale transaction**

fact\_sales preserves the POS data at its original transaction grain, joined against dim\_date (via the transaction date) and dim\_product (via the product id) to attach the appropriate surrogate keys. Alongside the transaction quantity, unit price, computed revenue, and hour of sale, a month\_key is carried directly on the fact table so that sales can be aggregated to a month without needing to join back through dim\_date for that purpose.

#### **9.2.2.4 fact\_payroll — one row per employee, per month**

fact\_payroll aggregates the payroll export to one row per employee per month, joined against dim\_employee to attach the employee surrogate key, and summarizing both the total amount paid and the number of individual payments made to that employee in that month. This grain — employee by month, rather than individual payment — matches how the business actually reasons about payroll cost: as a monthly line item per person, feeding directly into the payroll analysis view described in the next chapter.

## 10. Business Views

On top of the Gold-layer dimension and fact tables, SteadyFin defines ten SQL views inside the `cash_flow_gold` database. Each view answers one specific business question, and each view is a table that Power BI connects to directly as a data source. There is no extra step in between: Power BI is set up to read straight from these views, so whatever a view computes is exactly what appears on the dashboard, with no recalculation happening inside Power BI itself.

Because Power BI reads the views as-is, every percentage, ranking, and classification shown on any dashboard page is guaranteed to match what the view defines. This also means each dashboard page is built from one or more specific views — the table below shows exactly which view feeds which page.

| View                                       | Dashboard Page It Feeds                                   |
|--------------------------------------------|-----------------------------------------------------------|
| <code>vw_monthly_financial_overview</code> | Executive Overview                                        |
| <code>vw_quarterly_performance</code>      | Executive Overview                                        |
| <code>vw_yoy_comparison</code>             | Executive Overview                                        |
| <code>vw_cash_runway</code>                | Executive Overview                                        |
| <code>vw_business_kpi_summary</code>       | Executive Overview                                        |
| <code>vw_expense_breakdown</code>          | Payroll & Expenses                                        |
| <code>vw_credit_card_vendor_spend</code>   | Payroll & Expenses                                        |
| <code>vw_payroll_analysis</code>           | Payroll & Expenses                                        |
| <code>vw_sales_product_performance</code>  | Sales                                                     |
| <code>vw_shortage_detection</code>         | Business Health Monitor (and the AI Suggestions workflow) |

The Financial Insights page is the one exception: instead of reading a dedicated view, it connects directly to `fact_transactions` together with its dimension tables (`dim_date`, `dim_accounts`, `dim_categories`), because that page is built for free-form drill-down rather than a fixed set of figures.

## 10.1 Views Feeding the Executive Overview Page

### **vw\_monthly\_financial\_overview**

Revenue, total expenses, net cash flow, and closing balance for every month, plus the month-over-month percentage change in revenue and in expenses. This is the core trend view behind the Executive Overview page.

### **vw\_quarterly\_performance**

Monthly figures rolled up to the quarter level, with every quarter ranked by total revenue. This answers which quarter of the year performs best and which performs worst.

### **vw\_yoy\_comparison**

Revenue, expenses, net cash flow, and average payroll percentage of revenue, aggregated to one row per calendar year, with the year-over-year revenue growth rate between 2022 and 2023.

### **vw\_cash\_runway**

Closing balance divided by the trailing 3-month average of total expenses, expressed as a number of months. This tells the owner how long the business could operate if revenue stopped today.

### **vw\_business\_kpi\_summary**

The single landing view for the report: one row per month combining revenue, expenses, net cash flow, closing balance, payroll percentage of revenue, cash runway in months, gross point-of-sale revenue, and the month's shortage flag.

## 10.2 Views Feeding the Payroll & Expenses Page

### **vw\_expense\_breakdown**

For every month, the absolute amounts of COGS, operating expense, payroll, and credit card spend, plus each category's percentage share of that month's total expenses.

### **vw\_credit\_card\_vendor\_spend**

Credit card spend grouped by month, vendor, and category, excluding internal transfers made to pay off the card. This identifies exactly which vendor is driving spend in a given category and month.

### **vw\_payroll\_analysis**

Each employee's total pay for a month, shown alongside that month's revenue and what percentage of revenue the payment represents. The month's shortage flag is carried forward so payroll can be reviewed against the business's overall financial state.

## 10.3 View Feeding the Sales Page

### **vw\_sales\_product\_performance**

POS sales aggregated by month, product category, product type, and hour, with total revenue, units sold, transaction count, and average unit price. Each hour is bucketed into a readable time-of-day label — Morning, Lunch, Afternoon, or Evening — so the Sales page can show not just what sells, but when.

## 10.4 View Feeding the Business Health Monitor Page

### **vw\_shortage\_detection**

Every month's shortage flag and severity, its revenue, expenses, net cash flow, and closing balance, its 3-month trailing baselines, and its percentage deviation from those baselines. Each flag code is translated into a short, human-readable status message — for example, explaining that a WARNING means the balance has fallen below 1.5 times the recent average expenses. Rows are ordered by severity. This is the same view the Business Health Monitor page reads directly, and it is also the view the AI Suggestions workflow queries to identify which months need a generated recommendation — one view, feeding both a dashboard page and the AI step.

## 10.5 The Financial Insights Page

Unlike the other four pages, Financial Insights does not read a dedicated view. It connects directly to fact\_transactions, joined to dim\_date, dim\_accounts, and dim\_categories. A dashboard page built from a view can only show what that view was written to show; a page built to let a user freely drill from a yearly total down into months, then accounts, then categories needs access to the raw transaction rows themselves, not a pre-aggregated result. That is why this one page is the exception to the rule.

## 11. Power BI Dashboards

The Gold-layer views feed a five-page Power BI report: Executive Overview, Sales, Payroll & Expenses, Business Health Monitor, and Financial Insights. All five pages share the same navigation bar, so a user moves between them without losing context. Every visual reads directly from a Gold-layer view or fact table — no percentage, ranking, or category logic is written inside Power BI itself. The report only lays out and filters figures that the data engineering layer has already computed and validated.

### 11.1 Executive Overview Page

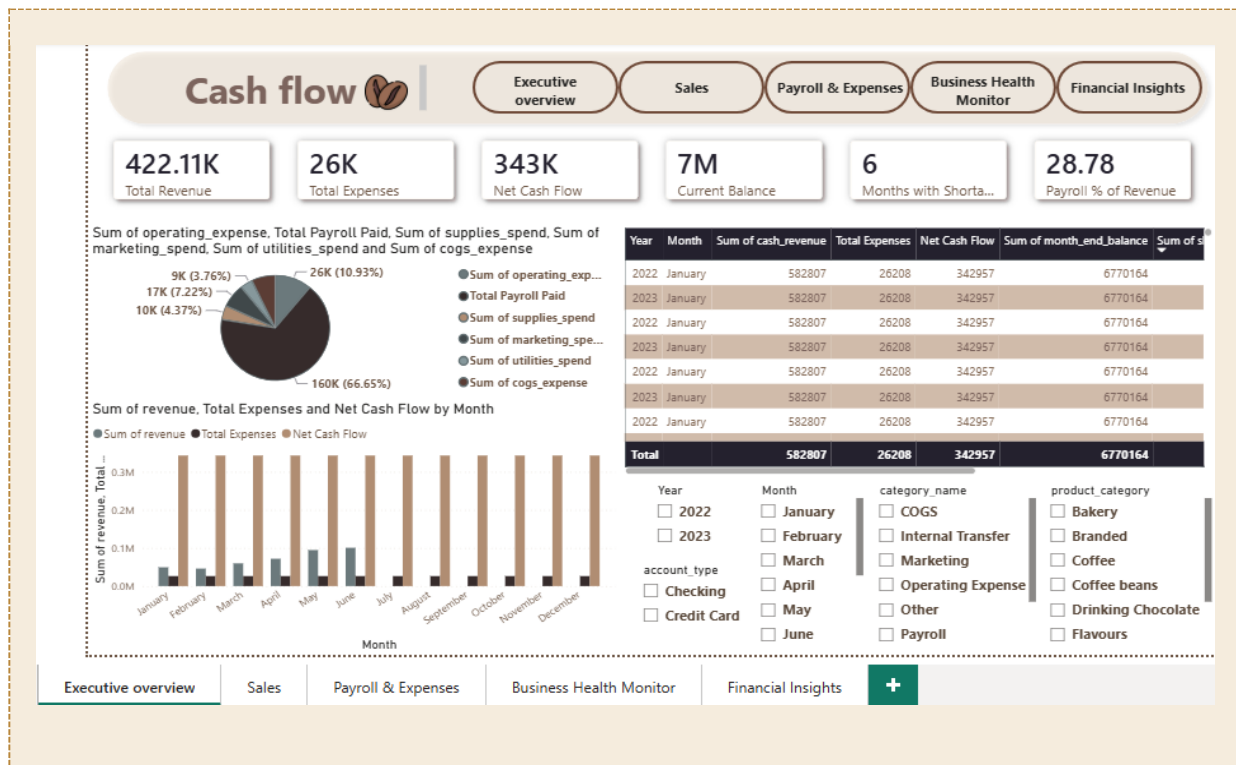


Figure 11.1 — Executive Overview dashboard page

Six KPI cards open the page: Total Revenue, Total Expenses, Net Cash Flow, Current Balance, Months with Shortage, and Payroll % of Revenue. A pie chart breaks expenses down by category — operating expense, payroll, supplies, marketing, utilities, and COGS — and a grouped bar chart tracks revenue, total expenses, and net cash flow month by month. A detail table lists year, month, revenue, expenses, net cash flow, and closing balance row by row, and the page is filterable by year, month, category, account type, and product category.

## 11.2 Sales Page

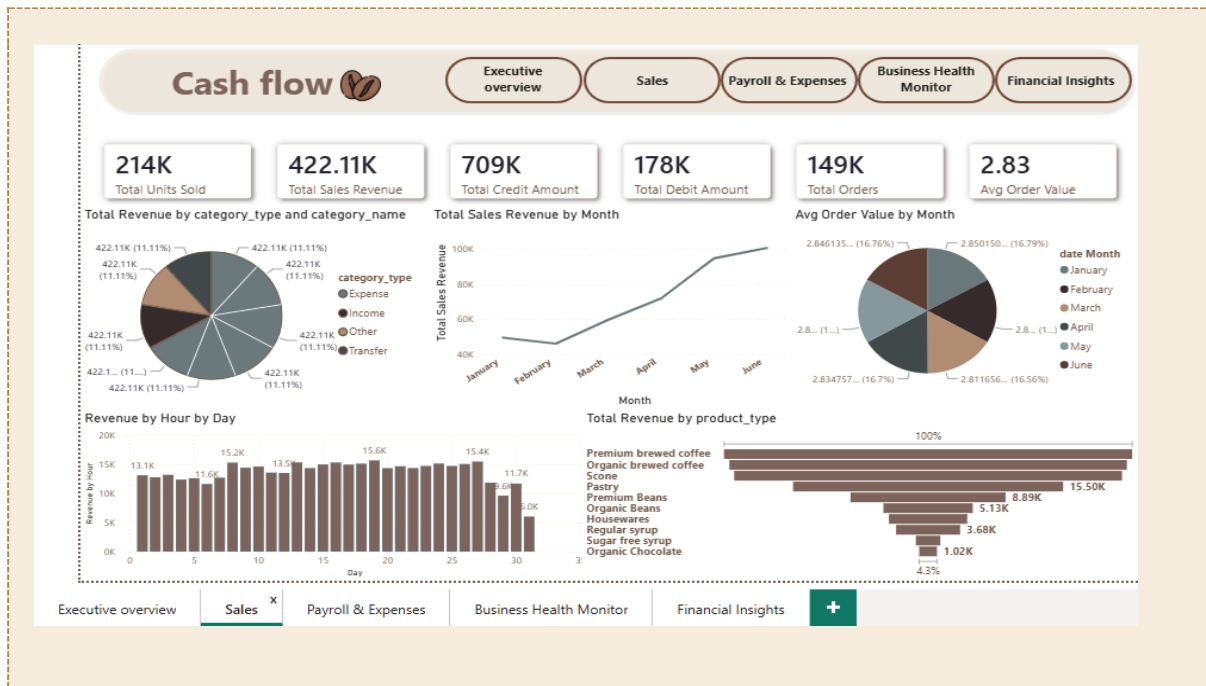


Figure 11.2 — Sales dashboard page

Six KPI cards summarize point-of-sale activity: Total Units Sold, Total Sales Revenue, Total Credit Amount, Total Debit Amount, Total Orders, and Average Order Value. A pie chart splits revenue by category type and category name, a line chart tracks total sales revenue by month, and a further pie chart shows average order value by month.

A bar chart of revenue by hour across the day shows exactly when sales peak, and a funnel-style bar chart ranks every product type by total revenue, from premium brewed coffee down to sugar-free syrup, showing which menu items carry the business and which are marginal.

## 11.3 Payroll & Expenses Page

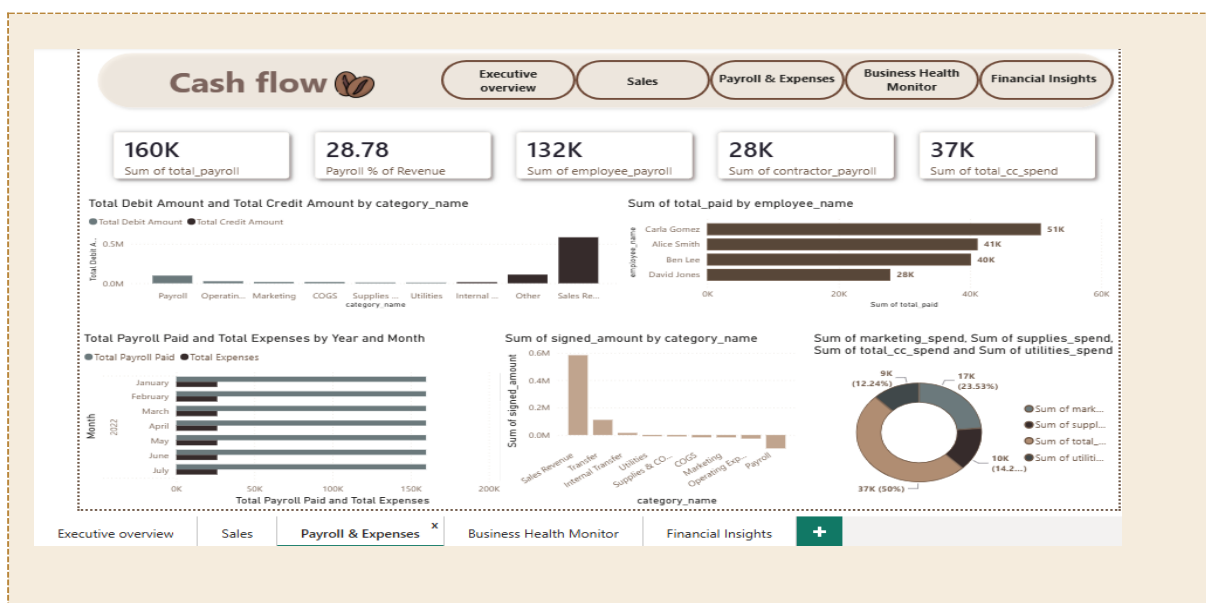


Figure 11.3 — Payroll & Expenses dashboard page



Five KPI cards cover the cost side of the business: Total Payroll, Payroll % of Revenue, Employee Payroll, Contractor Payroll, and Total Credit Card Spend. A bar chart compares total debit and credit amounts by category, and a second bar chart ranks employees by total amount paid, showing exactly how pay is distributed across the team.

A grouped bar chart compares total payroll paid against total expenses by year and month, a further bar chart shows the signed amount by category (led by Sales Revenue, followed by transfers, and each expense category below it), and a donut chart splits credit card spend into marketing, supplies, and utilities, isolating which discretionary categories drive the card bill.

## 11.4 Business Health Monitor Page

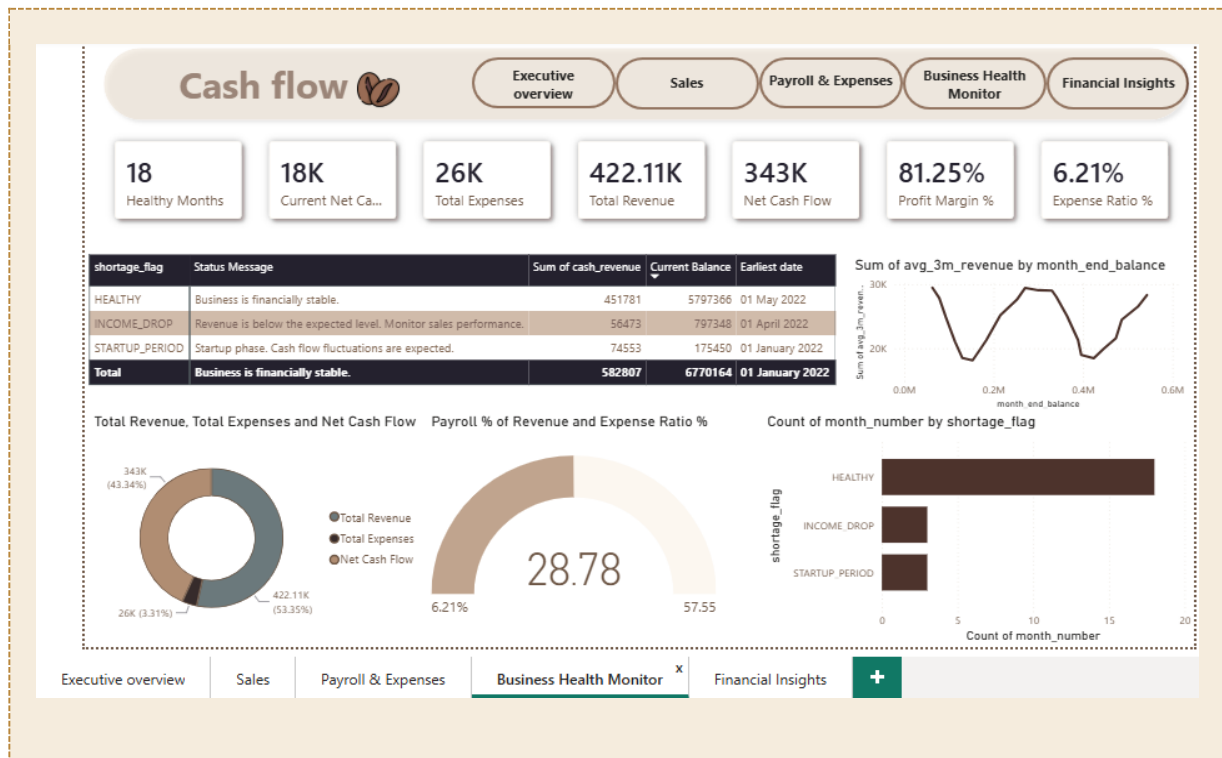


Figure 11.4 — Business Health Monitor dashboard page

This page is the dashboard-facing view of the shortage-detection logic, sourced from `vw_shortage_detection`. Seven KPI cards lead the page: Healthy Months, Current Net Cash, Total Expenses, Total Revenue, Net Cash Flow, Profit Margin %, and Expense Ratio %. A status table lists each shortage flag — HEALTHY, INCOME\_DROP, STARTUP\_PERIOD, and so on — next to its plain-language status message, its total revenue, its balance, and the earliest date that flag occurred.

A line chart tracks the 3-month rolling average revenue against month-end balance, a donut chart splits total revenue, expenses, and net cash flow, a gauge shows payroll percentage of revenue against the expense ratio, and a bar chart counts how many months fall under each shortage flag — giving a fast read on how many months of the business's history have been healthy versus flagged.

## 11.5 Financial Insights Page

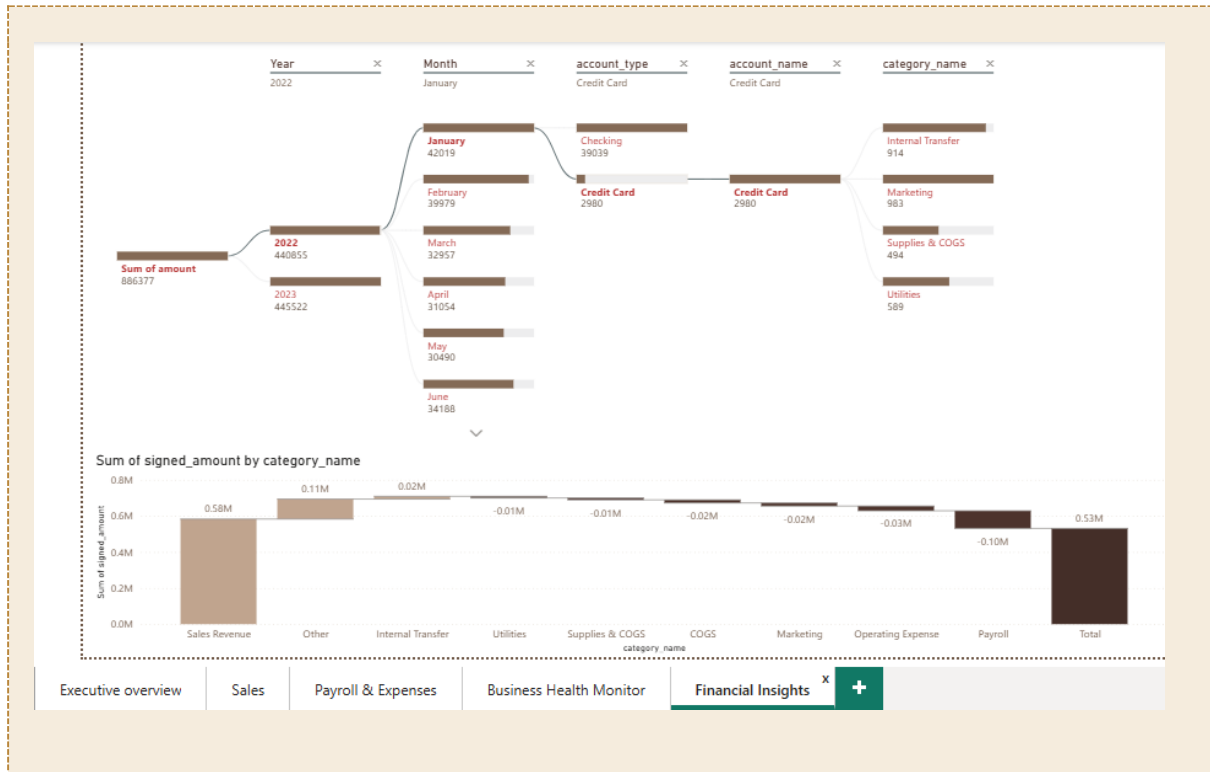


Figure 11.5 — Financial Insights dashboard page

This page is built for open-ended exploration rather than a fixed set of KPIs, using `fact_transactions` directly. A decomposition tree lets a user drill from total transaction amount down through year, month, account type, account name, and category, expanding whichever path they want to investigate — for example, splitting a year into months, then a month's credit card activity into its categories.

Below the decomposition tree, a waterfall chart shows the signed amount by category, starting from Sales Revenue and stepping down through the expense categories (COGS, marketing, operating expense, payroll, and so on) to arrive at the net total. Together, the two visuals give a single page where any transaction pattern can be traced from a high-level total down to its source without leaving the report.

## 12. AI Suggestions

The final stage of the SteadyFin pipeline closes the loop between automated detection and actionable guidance. Rather than simply flagging that a month looks unhealthy and leaving the interpretation to the business owner, the AI Suggestions workflow automatically generates a structured, data-grounded diagnosis and a prioritized set of recommendations for every month that the Gold layer has identified as needing attention.

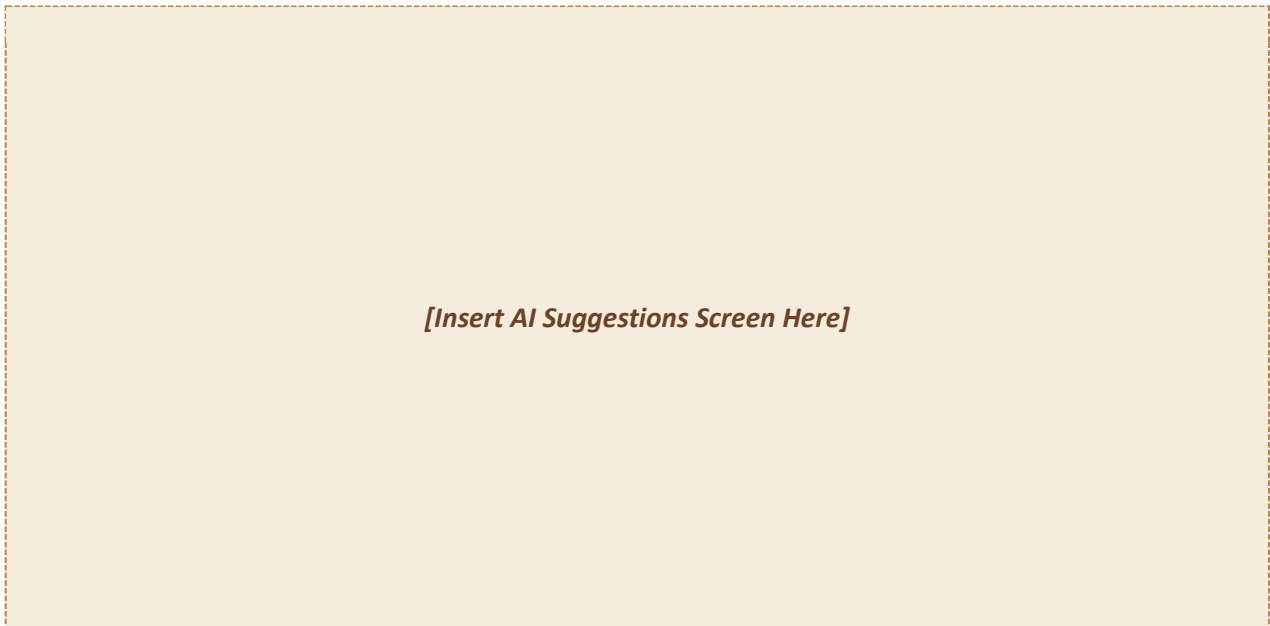


Figure 12.1 — AI-generated recommendation output

### 12.1 How the Workflow Identifies Which Months Need Analysis

The workflow begins by querying the `vw_shortage_detection` view, filtered to only the rows where `needs_ai_suggestion` is true — that is, every month classified as `CRITICAL`, `EXPENSE_SPIKE`, `INCOME_DROP`, or `WARNING`, ordered from most recent to oldest. If no month meets this condition, the workflow reports that all financial indicators are healthy and takes no further action, avoiding the wasted cost and noise of generating AI commentary for a business that currently has no problem to explain.

### 12.2 Assembling the Financial Context for Each Flagged Month

For every flagged month, the workflow does not send the language model a single number — it assembles the complete analytical picture of that month by querying six additional Gold-layer views, each contributing a different dimension of context:

- `vw_business_kpi_summary` — the executive-level KPI snapshot for the month (revenue, expenses, net cash flow, closing balance, payroll percentage, cash runway).
- `vw_monthly_financial_overview` — the month's position in the broader revenue and expense trend, including month-over-month change.
- `vw_expense_breakdown` — exactly how the month's spend was distributed across COGS, operating expense, payroll, and credit card categories.

- `vw_cash_runway` — how many months of buffer the business's current balance represents.
- `vw_credit_card_vendor_spend` — the top three vendors by spend for that month, so the model can name a specific vendor rather than speak generically about "credit card spend."
- `vw_sales_product_performance` — the top five selling products for the month, giving the model visibility into whether a revenue issue is tied to specific menu items or time-of-day patterns.
- `vw_payroll_analysis` — the top five payroll records for the month, so the model can reference specific employees, roles, or pay types if payroll is a relevant factor.

Each of these result sets is formatted as plain text and combined, together with the month's headline financial context (its shortage flag, its plain-language flag explanation, its revenue and expense figures, and its deviation from the three-month baseline), into a single, comprehensive prompt.

### 12.3 Prompting Strategy

The prompt explicitly positions the model as a senior Chief Financial Officer and business consultant specializing in coffee shop businesses, and gives it a three-step mandate: first analyze the available data, second identify the real root cause of the financial issue, and third recommend practical, data-supported actions. The instructions explicitly tell the model to use only the data provided, to avoid generic financial advice, to explain why the issue happened, and to reference the specific expense drivers, sales performance, payroll, and vendor spending behind it wherever relevant, with recommendations prioritized by business impact.

### 12.4 Technology Used

The AI Suggestions notebook uses Google's Gemini generative AI model through the `google-genai` Python client library, calling a Gemini model with its response format constrained to JSON. Business logic and data preparation for this step are performed entirely in Spark SQL against the Gold-layer views described above; the AI call itself is a single, well-scoped request per flagged month, keeping the language model's role limited strictly to interpretation and recommendation rather than any form of data processing.

### 12.5 What the Workflow Produces

For every flagged month, the workflow produces one structured recommendation object containing the month, the issue, the root cause, a priority level, the business impact, and a prioritized list of concrete recommendations — output that is explainable, specific to that month's actual data, and ready to be surfaced directly to a business owner or consultant through the reporting layer, rather than requiring further interpretation.

## 13. Website

The BI layer of SteadyFin's architecture includes a website channel, positioned alongside Power BI and the AI Suggestions workflow as one of the three ways the finished Gold-layer insights are ultimately surfaced to end users.

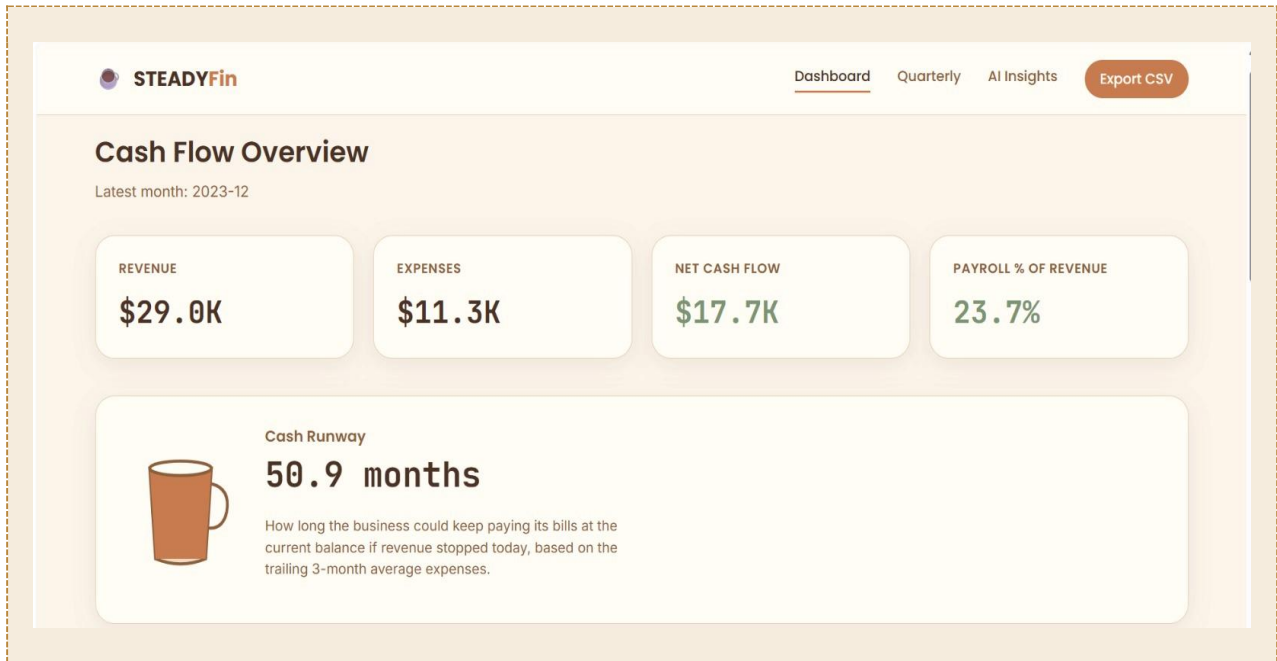


Figure 13.1 — SteadyFin website home page

The website's role in the architecture is to present the same governed, business-ready data and AI-generated recommendations that power the Power BI dashboards through a web-based channel, extending the reach of the pipeline's output beyond users who work directly inside Power BI. As with every other consumption channel in this project, the website reads from the finished Gold layer rather than performing its own independent data processing, preserving the single-source-of-truth principle that runs through the entire architecture.

## 14. Future Enhancements

---

- Extending the shared date-parsing helper and the Bronze ingestion notebook to support additional or newly onboarded source systems beyond the current five.
- Expanding the payroll data-quality hook (`silver_dq_flag`), currently a structural placeholder, into an active set of validation rules.
- Adding further fact tables to the existing galaxy schema — for example, a supplier-invoice or inventory process — by joining them to the dimensions that already exist.
- Surfacing the AI-generated recommendations directly inside the Power BI report or the website, rather than as a standalone notebook output.
- Extending the shortage-detection engine with additional classification states as more historical data accumulates and new patterns emerge.

## 15. Conclusion

---

SteadyFin demonstrates a complete, production-style data engineering lifecycle applied to a genuinely common small-business problem: reconciling cash flow across multiple disconnected financial and operational systems. By organizing the pipeline into a disciplined Bronze, Silver, and Gold medallion architecture, modelling the business-ready layer as a galaxy schema with conformed dimensions, and exposing that model through a set of purpose-built business views, the project ensures that every number seen by an end user — whether on a Power BI dashboard or in an AI-generated recommendation — traces back to a single, well-understood, and auditable transformation.

The addition of an automated, self-adjusting shortage-detection engine and a generative AI advisory layer grounded strictly in that engine's own Gold-layer data moves the system beyond passive reporting, toward genuinely proactive, explainable financial monitoring — directly addressing the lack of early warning and the lack of actionable guidance identified in the project's original problem statement.